Anaconda MCP User Stories

Product Requirements Document
Version 1.0 | January 2025

Story 1: Environment Creation from Natural Language

As a **data practitioner starting a new project**,
I want to **describe my project goals in plain language and have an appropriate conda environment created for me**,
So that **I can start coding immediately without researching which packages I need or memorizing conda CLI syntax**.

Marketing Narrative:
"Go from idea to working environment in one conversation. Describe what you're building, and Anaconda MCP handles the rest."

Acceptance Criteria

- ☐ User can describe a project goal (e.g., "I need to build a time series forecasting model") and receive a proposed environment with relevant packages
- ☐ Environment is created with a user-meaningful name (not auto-generated hash)
- ☐ Creation confirmation includes list of installed packages with versions
- ☐ User receives the activation command or next steps appropriate to their context

Edge Cases to Test

Scenario	Expected Behavior
Ambiguous request ("I need to do some data stuff")	Claude asks clarifying questions before proposing packages
Request includes packages that conflict	Claude identifies conflict, explains it, proposes alternatives
Request includes package not available in user's channels	Clear error message naming the package and suggesting alternatives
Environment name already exists	Prompt user to overwrite, rename, or cancel
No conda installation detected	Graceful failure with instructions to install conda/Anaconda

Story 2: Package Discovery Through Conversation

As a **practitioner who knows what problem I need to solve but not which packages exist**,
I want to **ask about capabilities and receive package recommendations from my available channels**,
So that **I can discover relevant tools without searching documentation or forums**.

Marketing Narrative:
"Stop Googling 'best Python package for X'. Ask Anaconda MCP and get recommendations from packages you're actually allowed to install."

Acceptance Criteria

- ☐ User can ask capability questions ("What packages can do geospatial visualization?")
- ☐ Recommendations are limited to packages available in user's configured channels
- ☐ Response includes brief explanation of why each package fits the use case
- ☐ User can request installation of recommended package(s) in follow-up message

Edge Cases to Test

Scenario	Expected Behavior
No packages in user's channels match the request	Acknowledge gap, suggest contacting admin if enterprise channel, or suggest conda-forge if permitted
Multiple packages serve same purpose	Present options with tradeoffs (e.g., "matplotlib for static plots, plotly for interactive")
User asks about package not in any channel	Clarify package exists but isn't available in their configured channels
Request is for proprietary/commercial package (e.g., CUDA)	Note licensing considerations alongside technical fit

Story 3: Environment Inspection and Explanation

As a **practitioner working with an existing environment**,
I want to **ask what's installed and get plain-language explanations of each package's purpose**,
So that **I can understand inherited or shared environments without manual investigation**.

Marketing Narrative:
"Inherited a project with 200 dependencies? Ask Anaconda MCP to explain what's in there and why."

Acceptance Criteria

- ☐ User can request summary of any environment they have access to
- ☐ Response groups packages by function (e.g., "Data manipulation", "Visualization", "ML frameworks")
- ☐ User can ask follow-up questions about specific packages
- ☐ Response identifies potential issues (outdated packages, known conflicts)

Edge Cases to Test

Scenario	Expected Behavior
Environment has 200+ packages	Summarize by category first, offer to detail specific categories on request
Environment contains pip-installed packages alongside conda	Acknowledge mixed package manager state, note potential risks
Environment is corrupted or in inconsistent state	Report the issue clearly, suggest resolution steps
User asks about environment they don't have access to	Permission error with clear explanation

Story 4: Guided Package Installation with Dependency Awareness

As a **practitioner adding capabilities to an existing environment**,
I want to **request a package and understand what dependencies will change before confirming**,
So that **I can avoid breaking my working environment**.

Marketing Narrative:
"No more crossing your fingers after conda install. See exactly what will change before you commit."

Acceptance Criteria

- ☐ User can request package installation in conversational language
- ☐ Response previews new packages, upgrades, and downgrades before execution
- ☐ User must confirm before changes are applied
- ☐ Post-installation confirmation summarizes what changed

Edge Cases to Test

Scenario	Expected Behavior
Installation would downgrade a critical package (e.g., Python itself)	Explicit warning with explanation of impact
Installation would take >5 minutes to solve	Inform user of expected wait time
Installation fails mid-process	Clear error state, guidance on rollback or retry
User requests package already installed	Confirm it's present, offer to upgrade if newer version available
User requests specific version that conflicts with environment	Explain conflict, suggest compatible version or new environment

Story 5: Environment Cleanup and Management

As a **practitioner with multiple environments accumulated over time**,
I want to **review my environments and remove unused ones through conversation**,
So that **I can reclaim disk space and reduce confusion without memorizing cleanup commands**.

Marketing Narrative:
"Spring cleaning for your conda environments. Review, archive, or remove with simple conversation."

Acceptance Criteria

- ☐ User can request list of all environments with basic metadata (size, last modified, package count)
- ☐ User can request deletion with confirmation step before execution
- ☐ Deletion confirmation names the environment explicitly to prevent accidents
- ☐ User receives confirmation of successful deletion with space reclaimed

Edge Cases to Test

Scenario	Expected Behavior
User requests deletion of base environment	Refuse with explanation that base cannot be deleted
User requests deletion of currently active environment	Warn user, require explicit confirmation
Environment deletion fails (permissions, file lock)	Clear error with suggested resolution
User asks to delete environment by partial name match with multiple hits	List matches, require user to specify

Story 6: Project-to-Environment Workflow

As a **practitioner who has planned a project in conversation with Claude**,
I want to **generate an environment that matches the technical decisions we discussed**,
So that **my planning work flows directly into a working development setup**.

Marketing Narrative:
"Your project plan becomes your environment. No lost context between planning and execution."

Acceptance Criteria

- ☐ After discussing project requirements, user can request environment creation referencing the conversation
- ☐ Environment includes packages aligned with decisions made in conversation (e.g., "we agreed on PyTorch, not TensorFlow")
- ☐ User can request an environment.yml export for reproducibility
- ☐ Environment is documented with comments or metadata linking to its purpose

Edge Cases to Test

Scenario	Expected Behavior
Conversation mentioned packages in passing but didn't confirm them	Ask for confirmation of ambiguous inclusions
Conversation spanned multiple sessions	Work with packages explicitly confirmed in current session, note any gaps
User asks for environment.yml but environment was created with pip packages	Note mixed state, export both conda and pip dependencies appropriately
Project requirements changed mid-conversation	Use most recent confirmed requirements, offer to review if unclear

Story 7: Enterprise Channel Compliance

As a **practitioner in an organization with approved package channels**,
I want to **install packages knowing they come from my organization's secure sources**,
So that **I remain compliant with security policies without manually checking provenance**.

Marketing Narrative:
"Enterprise-grade security without enterprise-grade friction. Anaconda MCP respects your organization's channels automatically."

Acceptance Criteria

- ☐ Package recommendations and installations use only user's configured channels
- ☐ When a package exists in public channels but not enterprise channels, user is informed clearly
- ☐ User can ask which channels are currently configured
- ☐ No packages are installed from channels not in user's configuration

Edge Cases to Test

Scenario	Expected Behavior
User explicitly requests conda-forge package when conda-forge isn't in their channels	Decline, explain channel restriction, suggest contacting admin
User's channel configuration is empty or corrupted	Graceful failure with diagnostic information
Package available in multiple configured channels at different versions	Use channel priority as configured, note if relevant
Enterprise channel is unreachable (network issue)	Clear error distinguishing network failure from package-not-found

Summary for Handoff

For Marketing

Stories 1, 2, 6, and 7 carry the strongest narrative weight. The throughline is: "Anaconda MCP removes the gap between knowing what you want to build and having the environment to build it, while keeping you compliant."

For Engineering/QA

Edge cases focus on the realistic failure modes that will drive support tickets if unhandled. Priority order for test coverage:

- **Dependency conflicts (Story 4)** - highest user frustration potential
- **Channel compliance (Story 7)** - highest enterprise risk
- **Environment state errors (Stories 3, 5)** - most common support issues

For PI Objectives

These stories map to the "Anaconda MCP" deliverable. Acceptance criteria are specific enough to define "done" for the Q1 release.